

Spark 原理详解 - 第 18 讲

Spark 性能优化：从 Spark UI 证据到可解释的调优

Partition · Shuffle · Skew · Join · Cache · Spill/GC · Small Files · Executor

本讲目标 学完后你不需要背几十个参数，而应该能够看到一个“跑得慢”的 Spark Job，先判断慢在哪里，再选择最小、最有依据的优化动作。

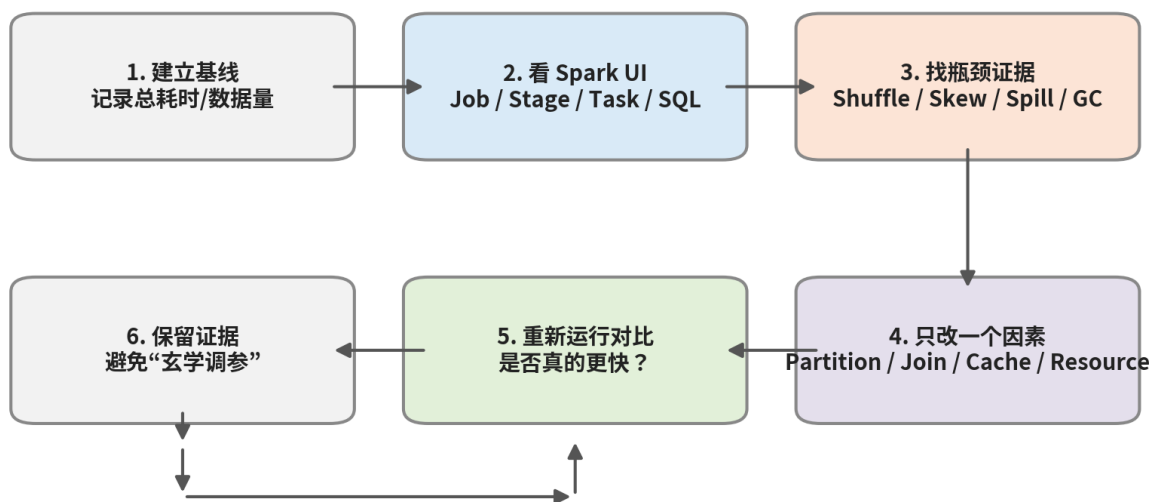
版本口径 本讲继续按 Spark 3.5.x 思维讲解。Spark 3.5.7 官方文档中，spark.sql.shuffle.partitions 默认值仍为 200，spark.sql.autoBroadcastJoinThreshold 默认 10 MB；AQE 默认开启，并可在运行时合并小 Shuffle 分区、调整 Join 策略和处理倾斜 Join。不要把这些默认值当成“所有任务的最佳值”。

看到的现象	第一反应应该查什么	不要先做什么
CPU 利用率低、Task 很少	Partition 数量 / 总 executor cores	盲目加 executor-memory
Shuffle Read/Write 巨大	Join/聚合方式、过滤是否太晚	只加机器
绝大多数 Task 完成，只剩 1~2 个	数据倾斜、热点 Key	单纯增加所有 Partition
Spill / GC 很高	单 Partition 工作集、缓存、Join/聚合	只提高 Driver 内存
输出几万个小文件	输出 Partition 数 / coalesce/repartition	把文件数直接压到 1

1. Spark 调优的第一原则：先找瓶颈，不要先改参数

“程序慢”只是一个结果，不是原因。Spark 的总耗时可能受输入扫描、CPU 计算、Shuffle 网络、磁盘 Spill、垃圾回收、数据倾斜、资源排队等不同因素支配。真正的调优，是找到当前最慢的那一项。

Spark 调优的正确顺序：先诊断，再动参数



核心：调优不是“把内存调大”，而是找到最慢的那一段系统成本。

图 1 证据驱动的 Spark 调优闭环

通俗比喻 汽车跑不快，可能是堵车、缺油、发动机、轮胎，也可能是司机走错路。Spark 调优也是一样：先看仪表盘和现场证据，再修真正的瓶颈。

可以把一个 Spark Job 的墙钟时间粗略理解成：

总耗时 $\approx$ 数据扫描 + 计算 + Shuffle/网络 + Spill/磁盘 + GC + 长尾 + 调度/排队开销

这不是严格数学公式，而是一张“诊断地图”。如果一个 Task 90% 时间都在 GC，那么优化 SQL 语法细节通常不是第一优先级；如果一个 Stage 只有 2 个 Task，而集群有 32 个 cores，那么再加内存也解决不了并行度不足。

2. Spark UI：调优时最重要的“证据现场”

你以后看 Spark UI，不要只看绿色进度条。至少建立四层观察顺序：SQL/Job → Stage → Task → Executor。

层级	重点看什么	典型问题
SQL / Jobs	哪个查询/Action 最慢；是否有 Exchange / Join	执行计划本身是否需要大 Shuffle
Stages	Stage Duration、Shuffle Read/Write、Input/Output	哪一个 Stage 占了主要时间
Tasks	Duration 分布、Input/Shuffle Records、Spill、GC Time	是否有长尾、倾斜、单 Partition 过大
Executors	Active Tasks、Memory、GC、Failed Tasks	资源是否均衡、是否有 Executor Lost / OOM

最有价值的比较 不要只看“平均 Task 10 秒”。要同时看中位数、最大值和分布。如果 199 个 Task 约 8 秒，只有 1 个 Task 600 秒，这不是“整体算力不足”，而是典型长尾/倾斜。

一个实用顺序：先找到最慢 Job，再找到最慢 Stage，再找到最慢 Task，最后对照该 Task 的 Shuffle Read、Input Size、Spill、GC Time。这样问题会从“整个 Spark 很慢”收缩成一个可解释的局部原因。

3. Partition 与并行度：不是越多越好，也不是越少越好

Spark 中一个 Stage 通常按 Partition 产生 Task。可同时运行多少 Task，又受到 executor cores 总数限制。因此 Partition 数量决定“有多少工作单元”，cores 决定“同一时间能跑多少工作单元”。

Partition 数量：太少、合适、太多都会付出代价



图 2 Partition 数量与并行度、调度开销之间的权衡

近似并发 Task 数 $\approx$ 所有可用 Executor Cores 的总和
 执行波次 $\approx \text{ceil}(\text{Stage 中 Task 数} / \text{可用 Cores})$

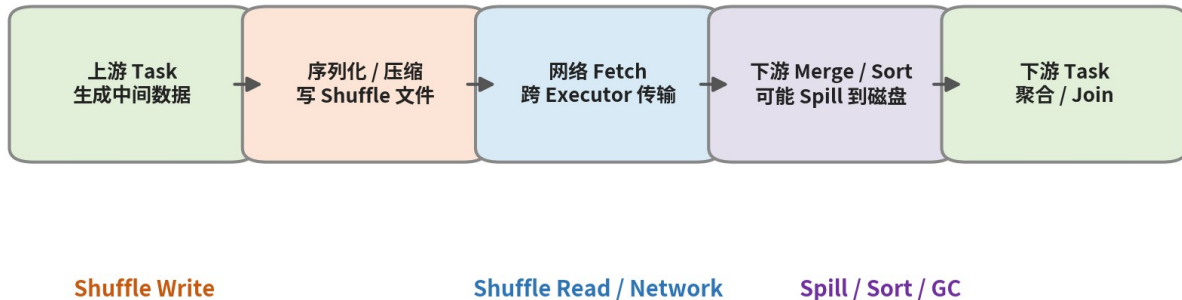
例子 集群有 8 个可用 cores。一个 Stage 只有 4 个 Partition，那么最多只有 4 个 Task 同时运行，另外 4 个 core 空闲；如果有 80,000 个极小 Partition，又会产生大量调度、序列化和小文件开销。

Spark 3.5.7 中，Spark SQL 的 `spark.sql.shuffle.partitions` 默认是 200。但 200 只是默认起点，不代表 2 GB 数据和 20 TB 数据都应该使用 200。AQE 可以在运行时合并过小的 Shuffle 分区，因此现代 Spark SQL 中不必把“手工算出一个完美分区数”当成唯一目标。

4. Shuffle：性能优化最常见的主战场

Shuffle 的本质是“数据根据新的分区规则重新分布”。它往往跨 Executor，需要写中间文件、网络传输、读取、合并/排序，并可能触发 Spill 和 GC。

Shuffle 为什么贵：不是一个动作，而是一串系统成本



优化原则：尽量让“更少的数据”进入 Shuffle，而不是只想着给 Shuffle 更多机器。

图 3 一次 Shuffle 背后的系统成本

- 能过滤的数据，尽量在 Shuffle 前过滤：让更少记录进入网络。
- 能只保留必要列，就尽量先 select 必要列：降低每条记录的宽度。
- 按 Key 聚合时，优先使用能进行局部预聚合的操作，而不是把所有 Value 原样聚到一起。
- 不要因为“看到 Exchange 就害怕”而消灭所有 Shuffle。有些 GroupBy、Join 本来就需要数据重分布；目标是减少不必要的 Shuffle 和 Shuffle 数据量。

最经典对比 如果目标是“每个 key 求和”，reduceByKey / DataFrame groupBy+sum 比先把同一 key 的所有 value 全部收集起来再求和更合理，因为前者可以在网络传输前做局部聚合。

5. Join：先判断“两边有多大”，再谈 Join 策略

Join 的性能差异往往不是来自 SQL 写法长短，而是来自物理执行策略。最重要的第一问是：两边都是大表，还是“大表 + 小表”？

Join 优化：大表×小表 与 大表×大表 是两类问题

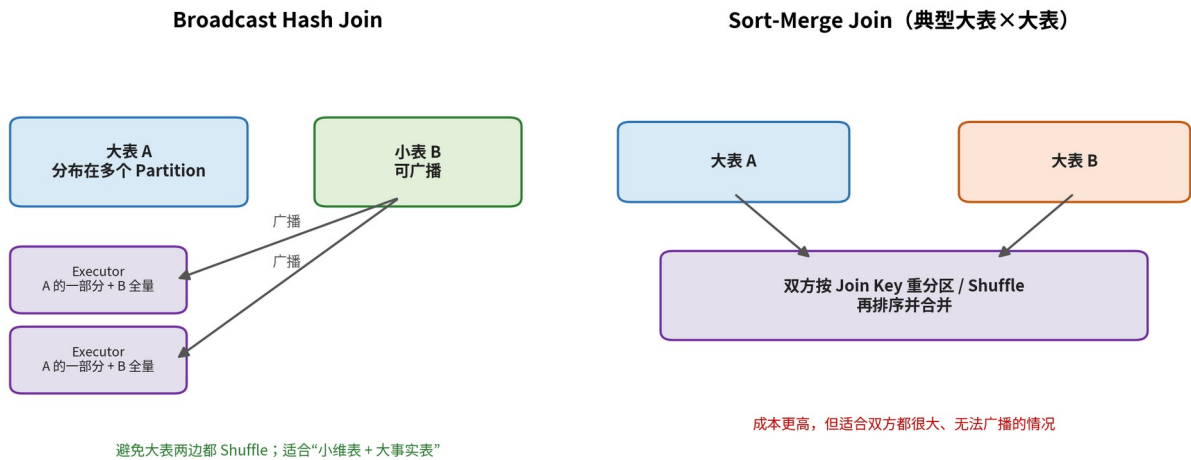


图 4 Broadcast Join 与大表 Join 的核心差异

场景	典型策略	核心成本
大事实表 + 小维表	Broadcast Hash Join	把小表复制到 Executor，避免大表两边都 Shuffle
大表 + 大表	Sort-Merge Join 常见	双方按 Join Key Shuffle，并排序/合并
统计信息不准	可能选错策略	需要看 explain / runtime statistics

Spark 3.5.7 中 `spark.sql.autoBroadcastJoinThreshold` 默认 10 MB。也可以使用 `broadcast hint`，但 `hint` 不是“绝对命令”，某些 Join 类型/计划仍可能无法采用指定策略。

```
from pyspark.sql.functions import broadcast

result = fact_df.join(broadcast(dim_df), "product_id", "left")
result.explain("formatted")
```

误区 “Broadcast Join 一定最快”是错的。小表如果实际上很大，广播会占用每个 Executor 的内存，甚至导致 OOM。是否广播应结合真实体积、统计信息和内存预算。

6. 数据倾斜：为什么 99% 很快，最后 1% 很久？

如果某个热点 Key 占据大量数据，那么 Hash 分区会把这个 Key 的数据集中到同一个目标 Partition。其他 Task 都完成后，这个超大 Partition 对应的 Task 仍然在运行，于是整个 Stage 被它拖住。

数据倾斜：总数据不一定大，但一个 Partition 可以拖死整个 Stage

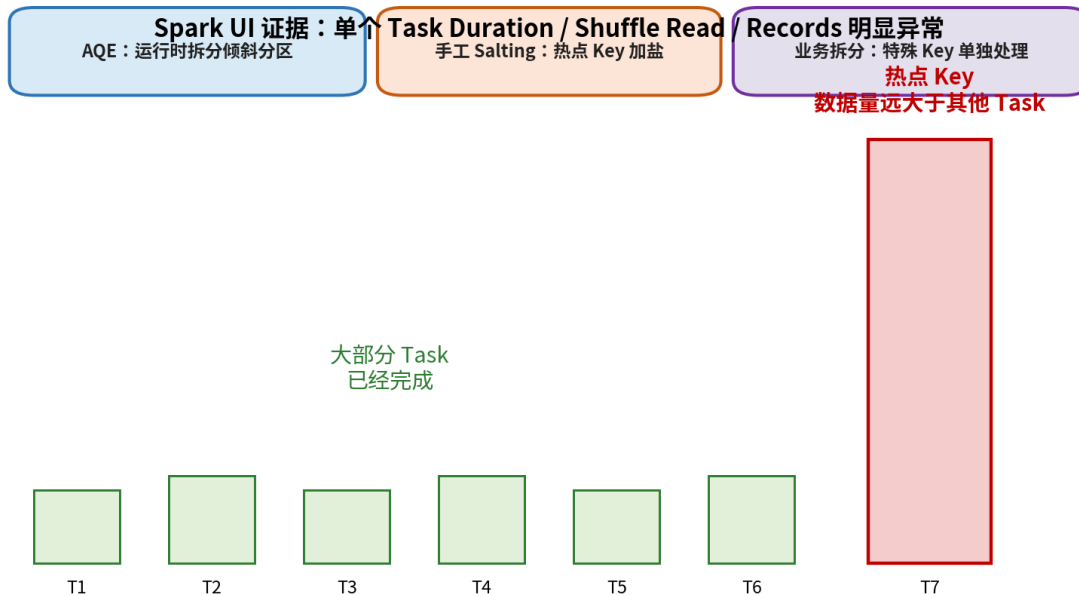


图 5 数据倾斜造成的长尾 Task

- 证据：Task Duration 最大值远高于中位数。
- 证据：某几个 Task 的 Shuffle Read / Records 远高于其他 Task。
- 证据：增加 Executor 后仍然只有同一个 Task 很慢。

Spark 3.5.x 的 AQE AQE 默认开启。Spark SQL 可以基于运行时统计合并小 Shuffle 分区，也可以对倾斜 Sort-Merge Join 的分区进行拆分/复制，以减少长尾。

如果 AQE 无法覆盖你的业务倾斜，还可以使用 Salting：把一个热点 Key 拆成多个“带盐 Key”先并行局部聚合，再去掉盐做第二次聚合。

```
# 思路示意：HOT 被拆成多个桶
HOT -> HOT_0, HOT_1, HOT_2, ...
第一阶段：按带盐 key 聚合
第二阶段：去掉盐，再按 HOT 聚合
```

7. Cache / Persist：只缓存“会被重复使用且重算昂贵”的数据

缓存的本质是“用存储空间换重新计算时间”。它不是自动加速器。如果一个 DataFrame 只用一次，cache 可能只有写缓存的成本，没有任何复用收益。

情况	是否考虑 Cache	原因
同一个中间 DataFrame 被多个 Action 反复使用	通常值得	避免重复扫描/Shuffle/复杂计算
只使用一次	通常不值得	写缓存本身有成本
数据非常大、内存紧张	谨慎	可能挤压执行内存，引发 Spill/GC
过滤后数据显著变小，后面多次复用	常较合适	缓存更小、更有复用价值

```
cached = cleaned_df.filter("amount > 0").cache()
cached.count()      # 第一次 Action: 真正计算并填充缓存
# 后续多个 Action 复用 cached
...
cached.unpersist()  # 不再需要时释放
```

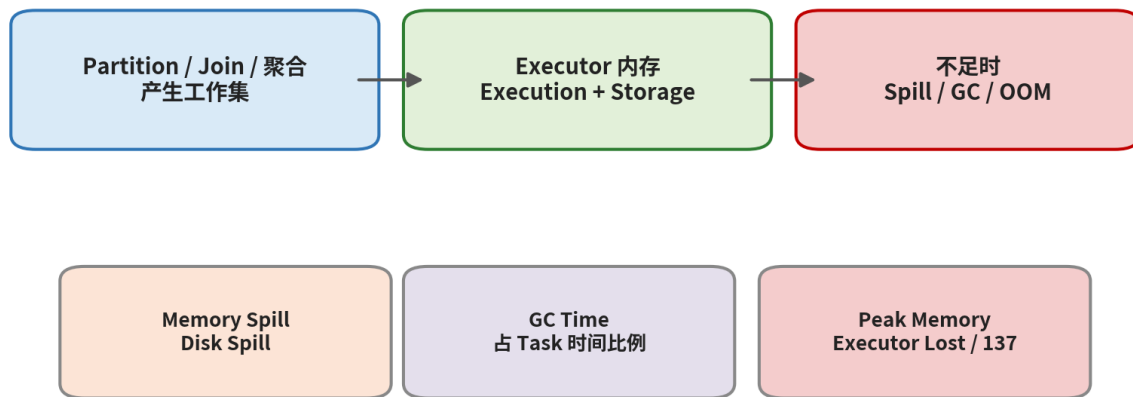
关键细节 cache() 只是“标记要缓存”，并不会立刻把数据装进内存。通常需要第一次 Action 才真正物化。

Spark SQL 对 DataFrame/Table 的缓存采用内存列式格式，并能扫描所需列、自动选择压缩，以降低内存占用和 GC 压力。

8. Spill、GC 与 OOM：不要把所有问题都归结为“内存不够”

当一个 Task 的工作集太大、聚合/Join 状态过大，或者缓存占用过多，Spark 可能先出现 Spill 和 GC 变重，然后才可能演化成 OOM 或 Executor Lost。

内存问题不要只看 OOM：Spill 和 GC 往往更早暴露



优先问：是不是单 Partition 太大、Join/聚合策略不合适、缓存过度？
“加内存”通常不是第一步。

图 6 内存压力从 Spill / GC 到 OOM 的演化

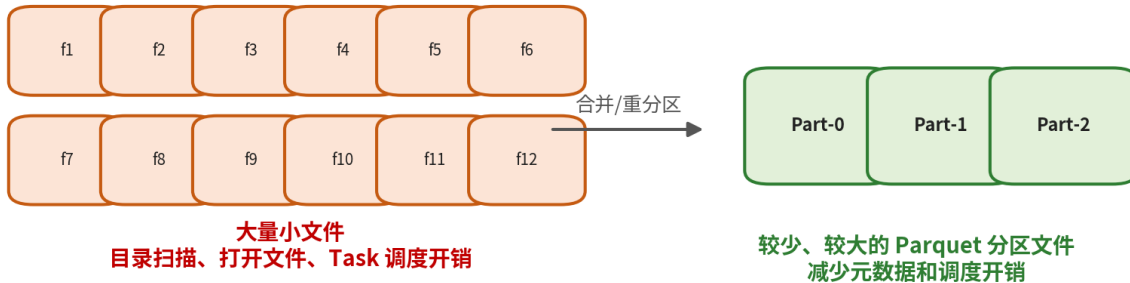
现象	更可能的原因	优先检查
Disk Spill 很高	单 Partition 数据太大、排序/聚合工作集大	Partition 大小、Shuffle、Join
GC Time 占比高	对象过多、内存压力、Python/JVM 边界开销	Task 数据量、缓存、序列化
Executor Lost / exit 137	容器内存超限/OOM 等	executor memory + overhead、日志
Driver OOM	collect/toPandas/大结果回传 Driver	是否把大数据拉回 Driver

重要区分 Driver 内存和 Executor 内存是两回事。Executor OOM 时把 `--driver-memory` 调大通常毫无帮助；`collect()` 导致 Driver OOM 时增加 Executor 数也解决不了根因。

9. 数据格式与小文件：很多“Spark 慢”其实发生在计算之前

如果 HDFS 上存在成千上万个很小的 CSV/JSON 文件，Spark 在读取阶段就会付出目录扫描、打开文件、生成输入 Partition 和调度大量小 Task 的成本。

小文件问题：数据总量不大，也可能因为“文件太碎”而很慢



注意：coalesce/repartition 也有成本；目标不是“文件越少越好”，而是避免极端碎片化。

图 7 小文件碎片化与较合理的分区文件

- 分析型数据优先考虑 Parquet：列式存储有利于列裁剪和谓词下推。
- 写出结果前根据后续使用方式调整 Partition 数，避免每个 Task 产生一个极小文件。
- coalesce(n) 常用于减少 Partition 数，通常比无条件 repartition(n) 少一次完整 Shuffle；但是否适用要看数据分布和目标。
- 不要为了“只有一个文件好看”长期使用 coalesce(1)：它可能把输出阶段压成单 Task。

Spark 3.5.7 文件读取参数 对文件数据源，spark.sql.files.maxPartitionBytes 默认 128 MB，用于控制读取时一个 Partition 最多打包多少字节；这只是一个重要起点，不等于每个 Task 必须正好 128 MB。

10. Executor 数、Cores 和 Memory：资源参数必须和工作负载匹配

第 17 讲已经学过：总可用 executor cores 大致决定并发 Task 上限。本讲把它用于调优：如果 Task 足够多但 CPU 经常空闲，要看资源是否不足；如果资源很多但 Task 数少，则应该先解决 Partition/Stage 并行度。

参数/资源	它主要影响什么	常见误区
--num-executors	有多少 Executor 进程	越多越快
--executor-cores	单 Executor 可并发多少普通 Task	单 Executor 给几十个 core 一定好
--executor-memory	Executor JVM heap	内存越大 GC 一定越少
memoryOverhead	JVM 外/网络/Python 等额外内存	只看 Xmx，不看 Container 总内存

为什么超大 Executor 也可能不好 一个 Executor cores 太多，会让大量 Task 共享同一 JVM、内存和 GC；同时 Executor 一旦丢失，影响更大。工程上通常追求“足够并发、不过度集中”的折中，而不是把整台节点塞进一个超大 Executor。

真正调资源时需要结合：节点总资源、YARN 队列限制、Task 类型（CPU/内存/IO）、Partition 数量、Python worker 开销。不要把网络上某个“万能 5 cores / 20 GB”公式直接套到所有集群。

11. 一套可以反复使用的 Spark 性能排查树

1. 先记录基线：输入数据量、总耗时、最慢 Job/Stage、Executor 数与 cores。
2. 如果 Stage Task 数很少：检查 Partition 数和并行度。
3. 如果 Shuffle Read/Write 很大：检查过滤/列裁剪是否太晚、聚合/Join 是否可减少数据。
4. 如果只有少数 Task 特别慢：检查数据倾斜和热点 Key。
5. 如果 Spill/GC 高：检查单 Partition 数据量、缓存、Join/聚合工作集，而不是只加内存。
6. 如果 Join 慢：看 Physical Plan，判断 BroadcastHashJoin / SortMergeJoin / Exchange 是否合理。
7. 如果输入/输出大量小文件：处理文件碎片和输出 Partition。
8. 如果 Executor Lost / OOM：区分 Driver 与 Executor，查看 YARN/Spark 日志和 memory overhead。
9. 每次只改变一个主要因素，重新运行并比较。

最低闭环 “我改了一个参数，程序变快了” 还不够。至少要能回答：原瓶颈是什么？哪条指标证明它？修改后哪条指标发生了预期变化？

12. 上机实验：用同一个任务观察 Partition、Shuffle 与 Join 策略

下面实验不追求“大规模”，而是训练你会看执行计划和 UI。数据量可以小，但要观察机制。

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName('PerfLab').getOrCreate()

# 1) 生成一批订单数据
orders = (spark.range(0, 1000000)
          .withColumn('product_id', (F.col('id') % 1000).cast('long'))
          .withColumn('amount', (F.col('id') % 100 + 1).cast('double')))

# 2) 观察聚合: 会出现 Exchange / Shuffle
agg = orders.groupBy('product_id').agg(F.sum('amount').alias('sales'))
agg.explain('formatted')
print('partitions =', agg.rdd.getNumPartitions())
print(agg.count())

# 3) 小维表 Join
products = spark.range(0, 1000).withColumnRenamed('id', 'product_id')
joined = orders.join(F.broadcast(products), 'product_id')
joined.explain('formatted')
print(joined.count())

spark.stop()
```

- 观察 groupBy 后 Physical Plan 是否出现 Exchange。
- 打开 Spark UI，看聚合 Stage 的 Shuffle Read / Write。
- 观察 Broadcast Join 计划中是否出现 BroadcastExchange / BroadcastHashJoin。
- 把 broadcast() 去掉，再比较 explain 和运行时间。
- 尝试 spark.conf.set("spark.sql.shuffle.partitions", 8)，比较 Task 数量；只用于实验，不要得出“8 永远最好”的结论。

单机虚拟机提示 你的教学虚拟机资源有限，所以本实验的目标是观察机制，不是做真实性能排名。真正性能结论必须在相同数据、相同资源、重复运行的条件下比较。

13. 综合案例：一个“慢销售分析 Job”应该怎么推理？

假设一个销售分析 Job 原来需要 28 分钟。Spark UI 显示：

观察证据	数值/现象
Stage A 扫描 Parquet	3 分钟
Stage B Join + 聚合	24 分钟
Stage B Task 中位数	35 秒
Stage B 最慢 Task	17 分钟
最慢 Task Shuffle Read	18 GB
其他 Task Shuffle Read	约 300~600 MB
GC / Spill	最慢 Task 显著更高

推理顺序：

10. 扫描只占 3 分钟，不是主要瓶颈。
11. 最慢 Task 17 分钟，而中位数 35 秒，存在严重长尾。
12. 最慢 Task 的 Shuffle Read 远高于其他 Task，强烈指向数据倾斜。
13. 先检查热点 product_id / region / null key，而不是先扩大所有 Executor。
14. 如果是 SQL Join，可观察 AQE skew join 是否生效；必要时对热点 Key 特殊拆分或 Salting。
15. 优化后再次比较 Task Duration 分布和最大 Shuffle Read，而不只比较总耗时。

这就是“会调优”和“会改参数”的区别 会改参数：看见慢就把内存从 4g 改成 8g。会调优：从 UI 证据推断为单 Partition 倾斜，针对热点 Key 改数据分布，再验证长尾是否消失。

14. 初学者最常见的 10 个调优误区

误区	为什么不对
1. Spark 慢就加内存	瓶颈可能是并行度、网络、倾斜或 IO
2. Partition 越多越快	太多会增加 Task 调度和小文件开销
3. 200 个 Shuffle Partition 是最佳值	它只是 Spark 3.5.7 默认值之一，AQE 还会动态调整
4. cache() 一定加速	只用一次的数据缓存反而有额外成本
5. collect() 只是普通 Action	它会把结果拉回 Driver，结果大时可能 OOM
6. Broadcast Join 总是最快	小表不够小时会带来 Executor 内存压力
7. 更多 Executor 一定解决倾斜	热点 Key 仍会落到同一个 Partition
8. Exchange 都应该消灭	有些 Join/聚合天然需要 Shuffle
9. 输出一个文件最方便	coalesce(1) 可能制造单 Task 瓶颈
10. 运行成功就代表性能合理	SUCCESS 只说明跑完，不说明资源利用和算法代价合理

15. 练习题与答案：把“现象”翻译成“瓶颈”

练习 1：集群有 16 个可用 cores，但某 Stage 只有 4 个 Task，CPU 使用率很低。第一检查项是什么？

答案 先检查该 Stage 的 Partition 数量/上游分区。此时最明显的问题是并行度不足，不应先增加 executor-memory。

练习 2：199 个 Task 都在 20~40 秒完成，一个 Task 跑了 12 分钟，并且该 Task 的 Shuffle Read 大几十倍。最可能是什么？

答案 数据倾斜/热点 Key。重点检查 Key 分布、AQE skew join、Salting 或特殊 Key 分流。

练习 3：同一个 DataFrame 只执行一次 count()，你为了“加速”先 cache()。一定更快吗？

答案 不一定。cache 需要把计算结果写入缓存，但没有第二次复用，可能只增加成本。

练习 4：10 TB 事实表 Join 一个 5 MB 维表。优先考虑哪类 Join？

答案 Broadcast Hash Join 是很自然的候选，因为小表可以复制到 Executor，避免大表双方 Shuffle；仍应通过 explain 和实际内存条件验证。

练习 5：你把输出 coalesce(1)，结果最后一个 Stage 特别慢。为什么？

答案 大量输出被压到一个 Partition/Task，形成单点瓶颈。应根据下游需要选择合理文件数，而不是为了“一个文件”牺牲并行度。

本讲最终记忆 Spark 性能优化的主线不是“参数越大越好”，而是：看 UI → 找最慢 Stage → 找异常 Task/Exchange → 判断 Partition / Shuffle / Skew / Join / Memory / IO → 做最小改动 → 再验证。

官方核对口径 (Spark 3.5.7)： Performance Tuning 与 Configuration 文档。版本相关默认值应以实际部署版本为准。